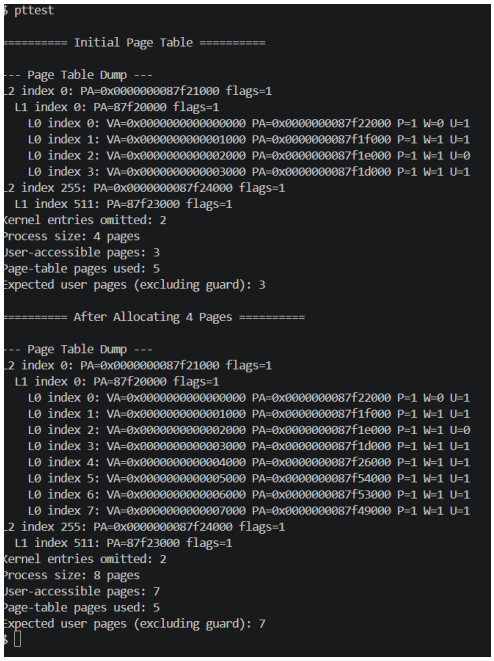
OS Lab Report

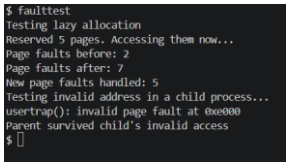
Question 1-Page-table inspection (vmprint)

Added a user-level test program (*pttest.c*) and connected it through the Makefile. The kernel-side page-table printing functionality and its system-call wiring allow the test to request a page-table dump. The output is used to inspect page-table levels and mappings, helping verify that the process page table is structured as expected.



Question 2 -Lazy allocation and page-fault counting

Changed *sys_sbrk()* so positive heap growth reserves virtual address space without immediately allocating physical pages. Updated virtual-memory routines to tolerate unmapped pages, added page-fault handling in *usertrap()*, and introduced a *getfaults()* system call plus a test program. The test reported 2 faults before accessing five reserved pages and 7 afterward—an increase of 5, matching the five pages accessed. A child process then triggered an invalid access at 0xe000; the kernel reported the fault and the parent survived, showing that the invalid access was isolated to the child.



Question 3 -FIFO and LRU page-replacement simulation

Created *user/pagerepl.c* and added it to the Makefile. The program simulates FIFO (evict the oldest loaded page) and LRU (evict the least recently used page), prints frame contents and hit/fault outcomes, and compares 3, 4, and 5 frames for the same 20-page reference string. With 3 frames, FIFO produced 15 faults and 5 hits (25% hit ratio), while LRU produced 12 faults and 8 hits (40%). For 3/4/5 frames, FIFO faults were 15/10/9 and LRU faults were 12/8/7. Thus, LRU performed better on this reference string, and increasing frame count reduced faults for both policies.

Conclusion. The tests demonstrate page-table inspection, demand allocation with fault counting, and the comparative behavior of FIFO and LRU. The reported counts match the expected 3-frame results for Question 3.

